Project's Report on

CI/CD Pipeline & Kubernetes Deployment for Multi-Agent Hospitality

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr No	Student Name	Enrolment
1	Aeshna Preyasi	EN22CS301075
2	Akash Ghosh	EN22CS301082
3	Aastha Gupta	EN22CS301019
4	Irifa Khaishagi	EN22CS301435
5	Aman Gaur	EN22CS301104
6	Adarsh Dibey	EN22CS301037

Group Name: Group 08D1

Project Number: DO-08

Industry Mentor Name: Aashruti Shah (Program Director, Datagami)

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-26

1 Problem Statement & Objectives

1.1 Problem Statement

In modern software engineering, deploying AI-driven applications manually leads to environment inconsistencies, deployment delays, and high margins for human error. Furthermore, traditional monolithic AI travel planners struggle with complex, multi-step reasoning. The absence of an automated CI/CD pipeline and container orchestration results in delayed code integration, downtime during updates, and the insecure handling of sensitive LLM API keys.

This project aims to solve these issues by designing a robust CI/CD pipeline for a Multi-Agent Hospitality System. By leveraging Docker, GitHub Actions, and Kubernetes, the project ensures seamless automation, reliable zero-downtime deployments, and highly structured AI outputs managed by CrewAI.

1.2 Project Objectives

The primary objectives of this project are:

- **To automate code integration using Git:** Implementing a trunk-based development approach where all updates pushed to the main branch are version-controlled and integrated efficiently.
- **To implement Continuous Integration (CI):** Automating the build process via GitHub Actions to package the application using an ubuntu-latest runner.
- **To containerize the application using Docker:** Packaging the Streamlit frontend, Python backend, and CrewAI logic into a portable, lightweight Docker image to ensure environment consistency.
- **To implement Continuous Deployment (CD):** Pushing the built image (aeshnapreyasi/travel-planner:latest) to Docker Hub and triggering automatic deployments via secure SSH to an AWS EC2 instance.
- **To deploy the application using Kubernetes:** Managing container orchestration using Minikube on EC2. Ensuring high availability, zero-downtime rolling updates, and secure API key management via Kubernetes Secrets.

- **To orchestrate a Multi-Agent AI Workflow:** Utilizing CrewAI to coordinate five specialized agents (Research, Planner, Budget, Writer, Reviewer) powered by the Groq LLM API.

1.3 Scope of the Project

The scope of this project includes:

- Development of a CI/CD pipeline triggered by GitHub pushes.
- Docker-based containerization of a Python/Streamlit web app.
- Deployment onto a Minikube Kubernetes cluster hosted on AWS EC2.
- Dynamic handling of Groq API keys using Kubernetes Secrets and ConfigMaps.
- Implementation of an Agentic AI architecture replacing a traditional backend database.

1.4 System Limitations:

- Limited to a single-node Kubernetes cluster (Minikube).
- Application is completely stateless; user history is not retained between sessions.
- No advanced cluster monitoring (e.g., Prometheus/Grafana) implemented initially.

2 Proposed Solution

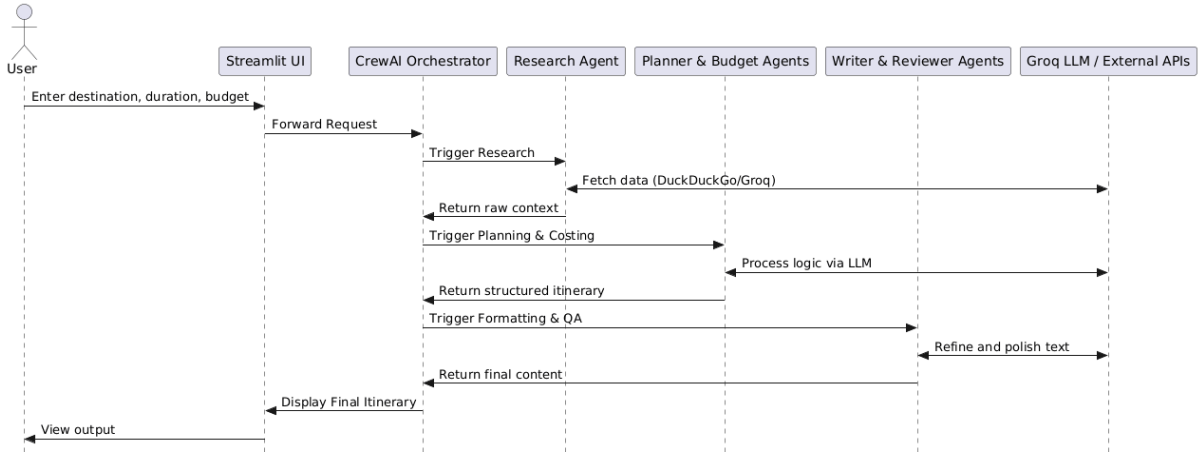
2.1 Key Features

- **Automated CI/CD Pipeline:** Triggered automatically on main branch pushes.
- **Zero-Downtime Deployments:** Achieved through Kubernetes rolling updates and imagePullPolicy: Always.
- **Secure Secrets Management:** Groq API keys are dynamically injected via Kubernetes Secrets, never hardcoded.
- **Agentic AI Workflow:** Sequential processing by Research, Planner, Budget, Writer, and Reviewer agents.
- **Stateless Containerized Architecture:** No database dependency, ensuring rapid scaling and simplified deployment.

2.2 Overall Architecture/Workflow

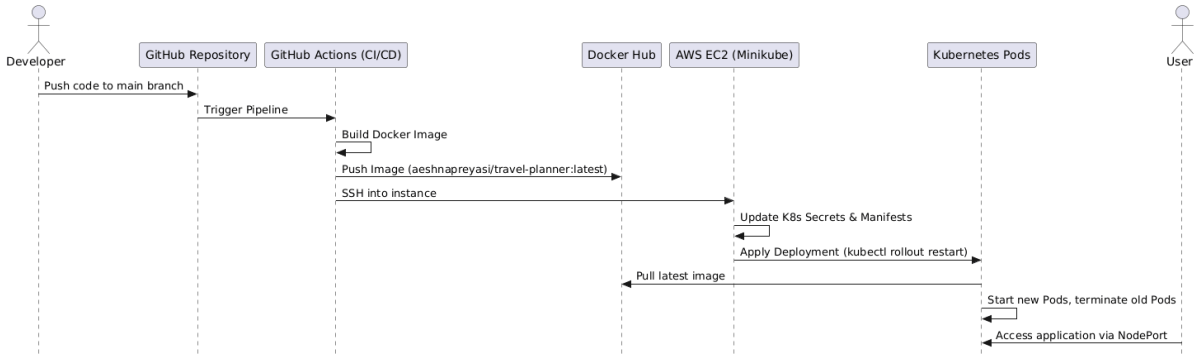
2.2.1 Application Workflow

1. **User Input:** The user accesses the Streamlit UI and inputs their destination, duration, and budget.
2. **Orchestration:** The request is received by the CrewAI Orchestrator.
3. **Research & LLM Processing:** The Research Agent gathers data via the DuckDuckGo API. Data is refined by the Groq LLM.
4. **Agent Pipeline Execution:** The Planner Agent drafts the itinerary, the Budget Agent estimates costs, and the Writer Agent formats the content.
5. **Review & Delivery:** The Reviewer Agent performs a final quality check before the Streamlit UI renders the complete travel plan to the user.



2.2.2 DevOps Workflow

1. **Code Commit:** Developer writes code and pushes to the GitHub main branch.
2. **CI Pipeline Initiation:** GitHub Actions runner initializes and checks out the repository.
3. **Docker Build & Push:** The Docker image is built and pushed to Docker Hub (aeshnapreyasi/travel-planner:latest).
4. **CD Pipeline Initiation:** GitHub Actions establishes an SSH connection to the AWS EC2 instance.
5. **Kubernetes Deployment:** K8s Secrets are updated, deployment manifests are applied, and Pods undergo a rolling restart to pull the newest image.

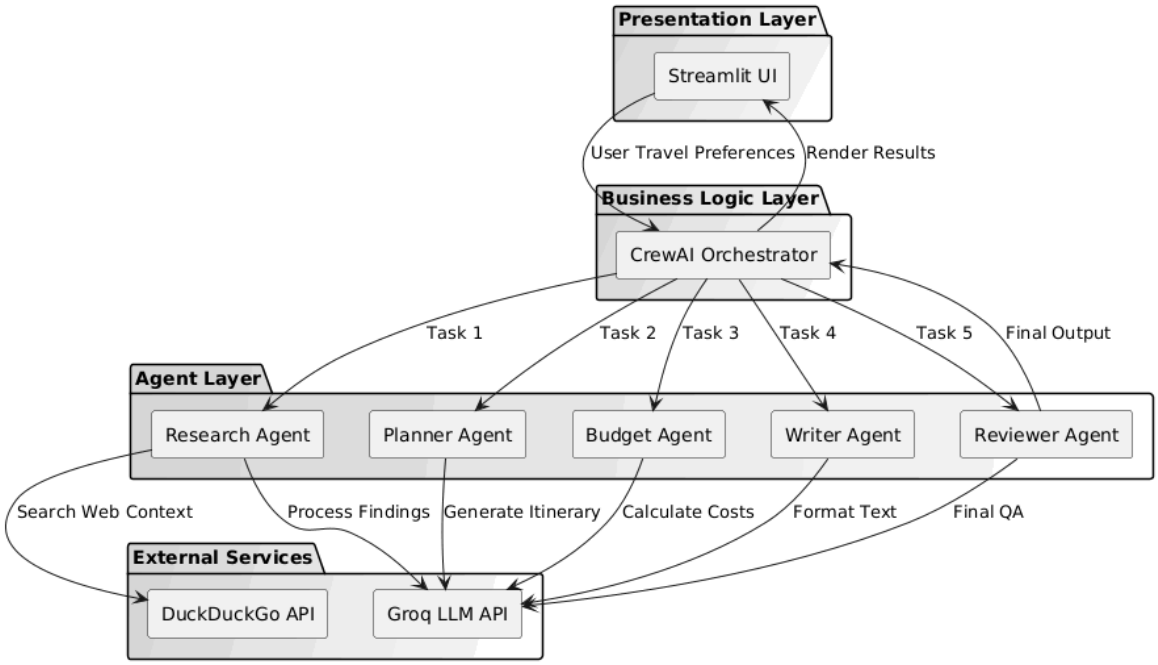


2.2.3 Architecture Explanation

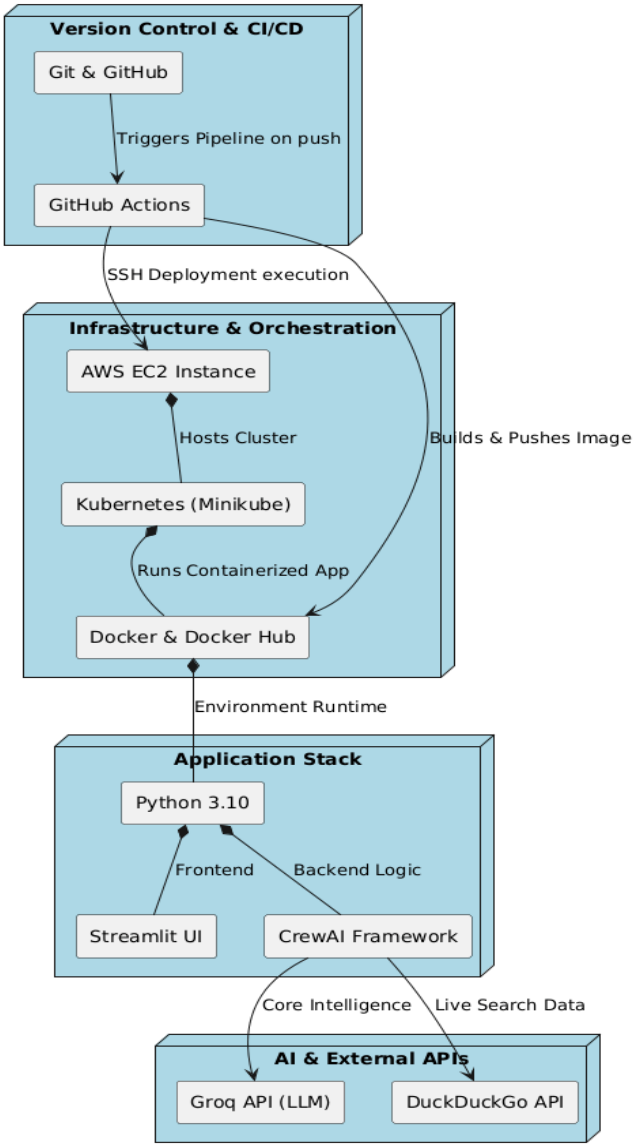
The system follows a stateless, container-based architecture. Instead of a traditional Microservice + Database model, the intelligence is outsourced to the Groq API. Docker ensures that the environment mapping between the developer's machine and the AWS EC2 instance is identical. Kubernetes manages the lifecycle of the Streamlit application container, ensuring self-healing if a pod crashes and managing external access via NodePort services.

2.2.4 Component Architecture Diagram

This diagram breaks down the application into its logical layers (Presentation, Business Logic, Agents, and External APIs), showing exactly how the pieces of your hospitality system talk to each other.



2.3 Tools & Technologies Used

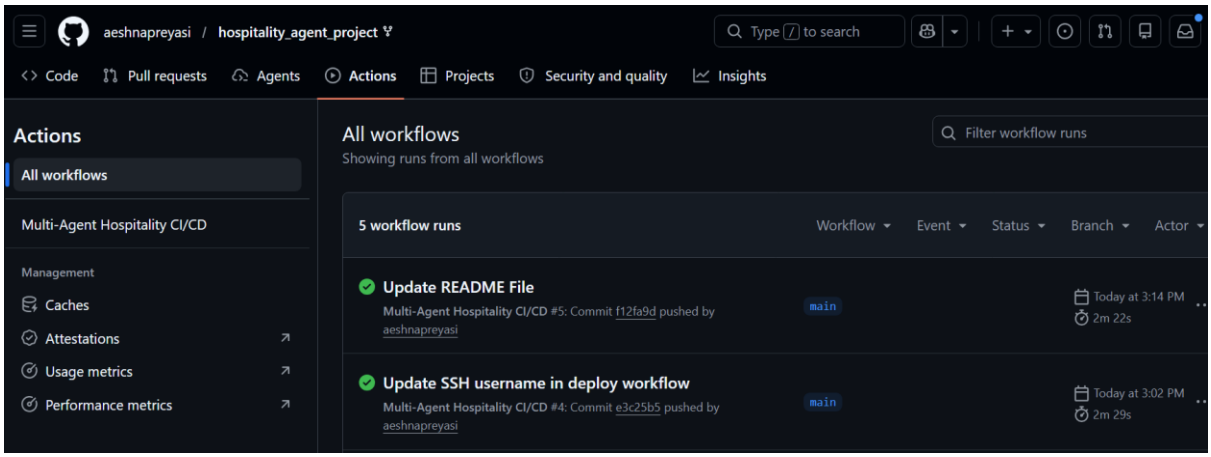


- **Git & GitHub:** Version control and trunk-based repository management.
- **Docker & Docker Hub:** Containerization and image registry.
- **GitHub Actions:** CI/CD pipeline automation.
- **Kubernetes (Minikube):** Container orchestration, Secrets management, and self-healing.
- **AWS EC2:** Cloud hosting infrastructure.
- **Python (Streamlit):** Web UI and presentation layer.
- **CrewAI:** Multi-agent orchestration framework.
- **Groq LLM API & DuckDuckGo API:** External AI and search intelligence.

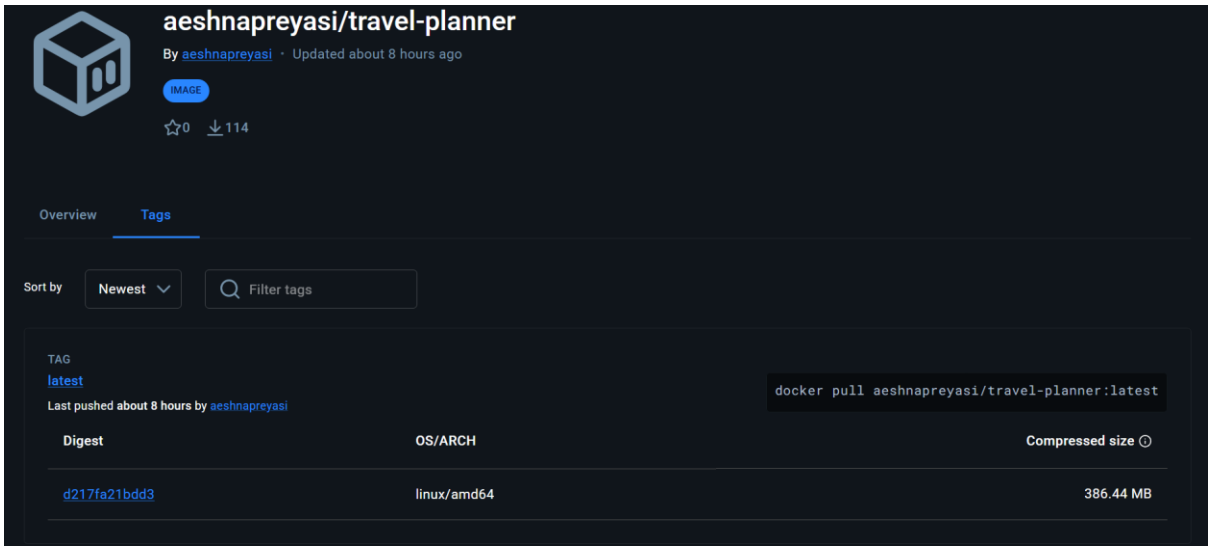
3 Results & Output

3.1 Screenshots / outputs

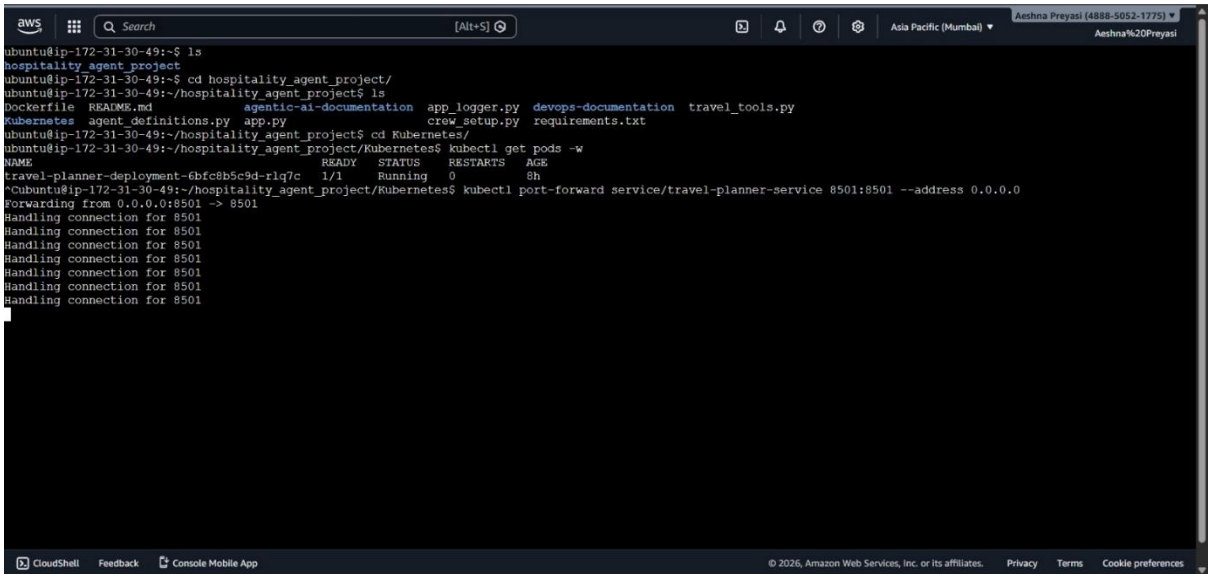
3.1.1 Figure 1: GitHub Actions CI/CD Pipeline Success



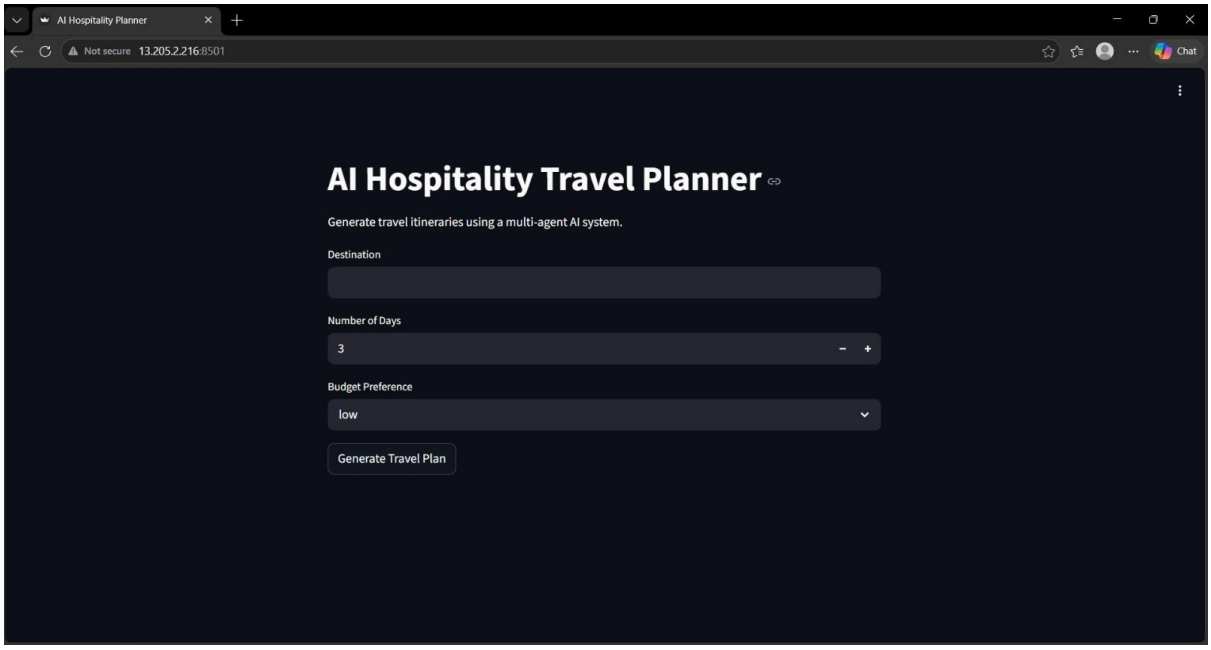
3.1.2 Figure 2: Docker Hub Image Registry



3.1.3 Figure 3: Kubernetes Pods Running on EC2



3.1.4 Figure 4: Application Running (Streamlit UI)



3.2 Key outcomes

- **Successfully Implemented CI/CD:** Code changes are now automatically built and deployed without manual intervention.
- **Achieved Zero-Downtime Deployments:** Kubernetes rolling updates allow the application to update its image without disrupting user access.

- **Secured Credentials:** Sensitive Groq API keys are effectively isolated from the codebase using Kubernetes Secrets.
- **Operational Agent Pipeline:** CrewAI successfully orchestrates five distinct agents to generate comprehensive, budget-aware travel itineraries.

4 Conclusion

The implementation of the CI/CD pipeline and Kubernetes deployment for the Multi-Agent Hospitality System successfully demonstrates a complete, modern DevOps lifecycle. By integrating GitHub Actions, Docker, and Kubernetes, the project achieved efficient automation, reliable scaling, and secure configuration management. The system highlights the viability of combining Agentic AI (CrewAI) with cloud-native deployment practices, significantly reducing manual deployment errors and creating a highly portable, resilient application environment.

5 Future Scope & Enhancements

- **Cluster Monitoring:** Integration of Prometheus and Grafana for real-time tracking of pod health and API latency.
- **Cloud-Native Migration:** Transitioning from Minikube on EC2 to a fully managed Kubernetes service like AWS EKS.
- **Advanced Deployment Strategies:** Implementing Blue-Green or Canary deployments for safer feature rollouts.
- **State Management:** Introducing a database (e.g., PostgreSQL or MongoDB) and Kubernetes Persistent Volumes to save user itineraries and account histories.
- **Auto-Scaling:** Implementing the Kubernetes Horizontal Pod Autoscaler (HPA) to handle traffic spikes dynamically.